

# Construyendo la Capa de Entrega: Express y APIs REST

Dejando atrás la gestión manual para dominar el enrutamiento, los middlewares y la semántica HTTP.

Gemini Notebook

# La ruta hacia una API estructurada

El Modelo Express

Patrones y Trade-offs

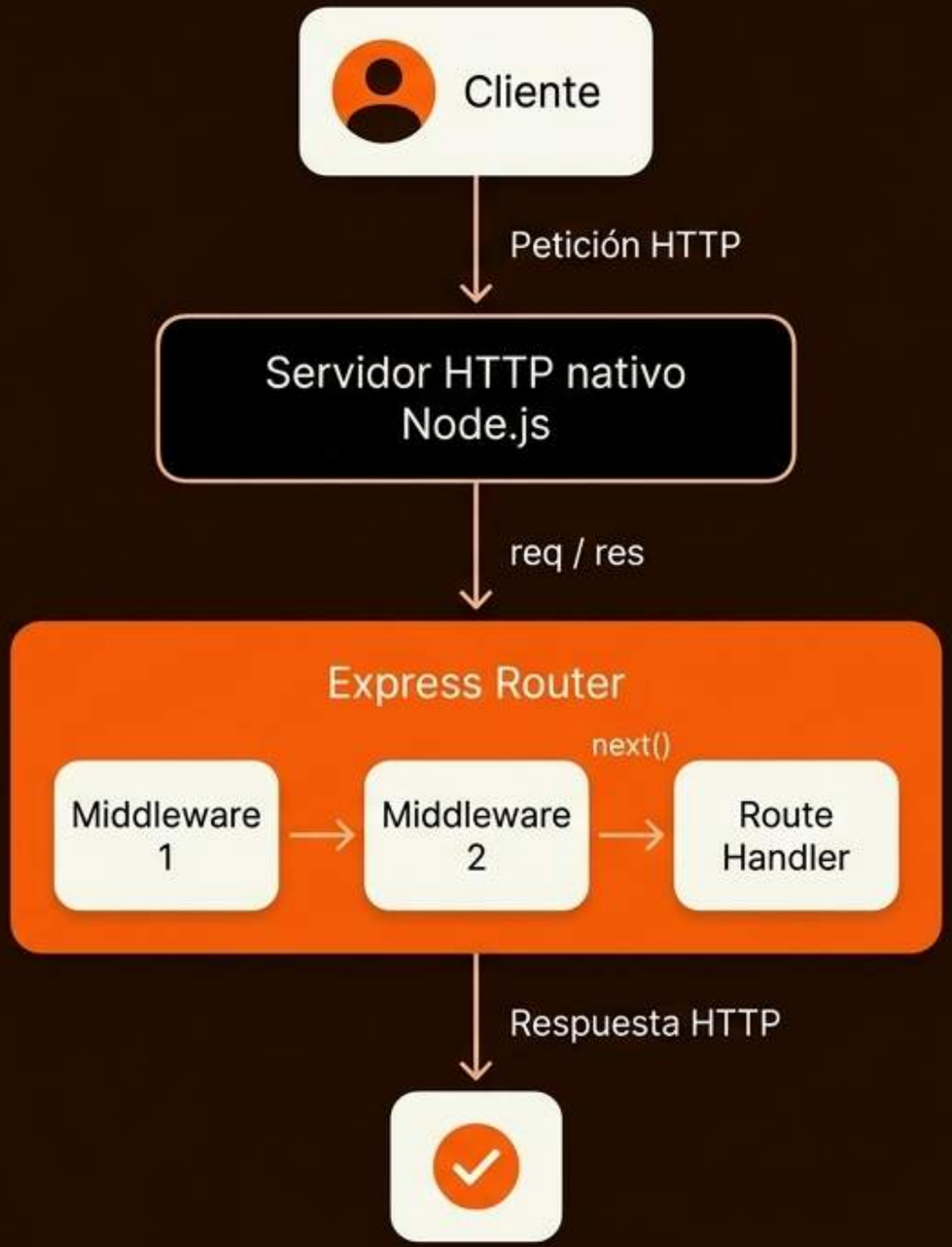
Diseño RESTful

Contrato Semántico

Gemini Notebook

# Express: Una capa de abstracción, no un reemplazo

- Construido sobre el servidor HTTP nativo de Node.js.
- Estandariza req (petición estructurada) y res (constructor de respuesta).
- El núcleo funcional es una cadena de procesamiento secuencial.




Gemini Notebook

# Analizando nuestro stack: ¿Por qué Express?

| ENFOQUE | VENTAJA CLAVE | RIESGO / TRADE-OFF |
| --- | --- | --- |
| **Servidor Nativo (Node.js HTTP)** | Cero dependencias, control total. | Parseo y enrutamiento manual. Alto costo de mantenimiento. |
| **Express (El estándar)** | Ecosistema maduro, middlewares componibles. | Flexibilidad requiere convenciones estrictas para evitar el caos. |
| **Opinionados (Ej: NestJS)** | Estructura predecible, convenciones fuertes. | Menor libertad, posible complejidad innecesaria. |

Gemini Notebook

# Arquitectura REST: Modelando recursos, no acciones

- REST (Representational State Transfer) expone entidades del dominio del negocio.
- Un recurso es un sustantivo, no un verbo. No es una tabla de BD.
- Interfaz uniforme: sin estado de sesión en el servidor.




Gemini Notebook

# Diseccionando la Petición: ¿De dónde extraemos los datos?

## req.params

*¿Qué recurso exacto?*

URI estable para identificar (Ej: /api/services/:sid). **Validar siempre el formato del ID.**

## req.query

*¿Cómo lo presento?*

Filtros, paginación u orden (Ej: ?page=2). **No modifica la identidad del recurso.**

## req.body

*¿Qué datos envío?*

Payloads estructurados para crear/actualizar (POST/PUT). Regla de oro: Jamás confiar en estos datos sin validar.

Gemini Notebook

# Buenas prácticas para una base de código escalable

- **Tratar Express** como una capa de entrega HTTP, no como el core del negocio.
- **Separar responsabilidades**: Rutas → Controladores → Servicios → Datos.
- **Controladores delgados**: Reciben la petición, delegan la lógica y responden.




Gemini Notebook

# Códigos HTTP: El contrato público de tu API

Permiten que proxies, SDKs y clientes interpreten el resultado sin conocer tu lógica interna.

**[ ÉXITO (2xx) ]**

- 🟢 **201 Created**: Recurso creado (idealmente devuelve identificador).
- 🟢 **200 OK**: Respuesta estándar estructurada.

**[ ERROR DEL CLIENTE (4xx) ]**

- 🟠 **400 Bad Request**: Fallo en validación de entrada (params/body).
- 🟠 **404 Not Found**: El recurso identificado no existe.

**[ ERROR DEL SERVIDOR (5xx) ]**

- 🔴 **500 Internal Error:** Fallo interno no anticipado. JAMÁS exponer stack traces.

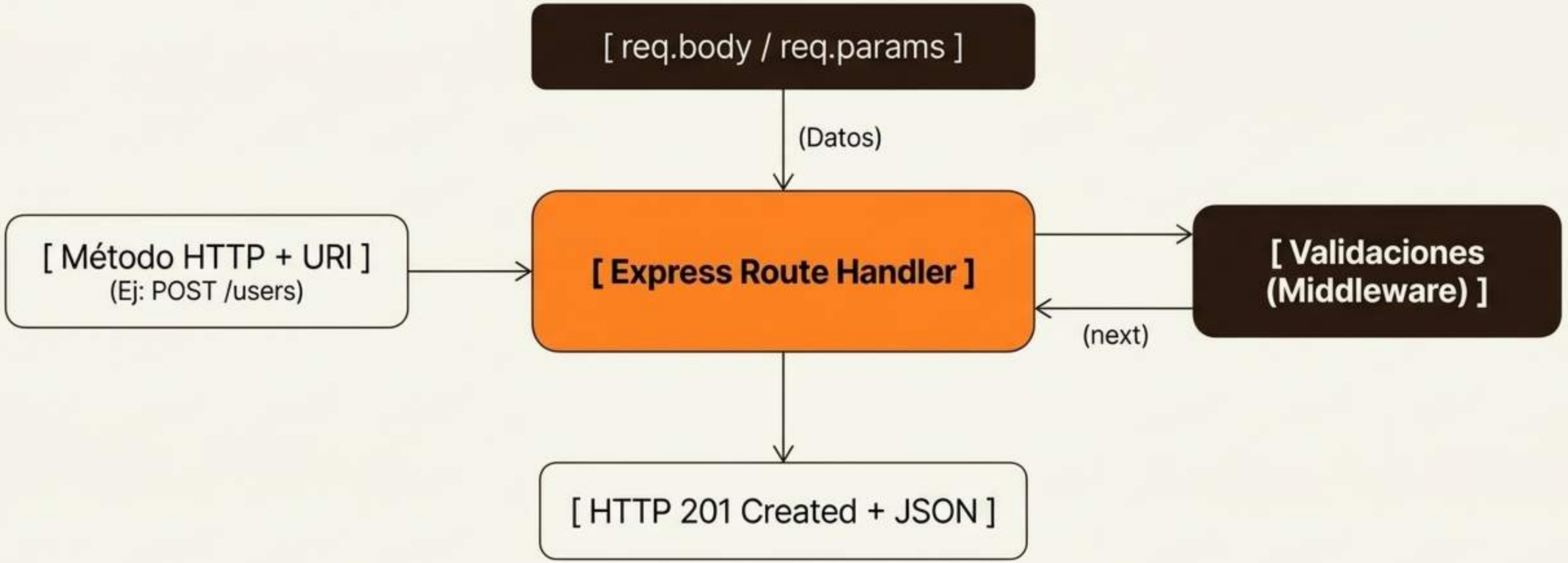
Gemini Notebook

# Anti-patrones que arruinarán tu API

- ❌ Devolver 200 OK pero incluir un “error: true” en el JSON.
- ❌ Usar 500 para errores predecibles de validación de negocio.
- ❌ Devolver tablas de bases de datos completas en lugar de entidades modeladas (/bookings_table).
- ❌ Inventar significados incompatibles para códigos estándar.



# Síntesis: El ciclo de vida de una petición HTTP




*El estado viaja en la solicitud, la intención en el método, el resultado en el código.*

Gemini Notebook